

Page Rank Homework

Fabi Pietro,

Summary

1. Algorithm
   - Power Method
   - Actual code
   - Explanation
2. Results
   - Graph 1
   - Graph 2
   - Hollins Database
   - Considerations
3. Exercises
   - Notes
   - Exercises

Inputs

- A: positive column-stochastic matrix.
- $v^{(0)}$: initial guess of the eigenvector.
- MaxIter: maximum number of iteration.
- RelTol: relative tollerance.

Main Code

1. $v^{(0)} = \frac{v^{(0)}}{\|v^{(0)}\|_2}$
2. $\lambda^{(0)} = \infty$
3. m = 0
4. do
5. $\quad \tilde{v}^{(m+1)} = Av^{(m)}$
6. $\quad \lambda^{(m+1)} = v^{(m)} \cdot \tilde{v}^{(m+1)}$
7. $\quad v^{(m+1)} = \frac{\tilde{v}^{(m+1)}}{\|\tilde{v}^{(m+1)}\|_2}$
8. $\quad m = m + 1$
9. while $(m \leq MaxIter)$ and $(|\lambda^{(m+1)} - \lambda^{(m)}| \geq RelTol \cdot |\lambda^{(m+1)}|)$

Outputs

- **Outputs**:
  - λ_1: first eigenvalue of A, should be equals to 1.
  - v: the eigenvector associated to λ_1, also the importance score of the pages.
- **Computational cost**:
  - **A full**: then Av will be $O(n^2)$.
  - **A sparse**: then Av is $O(n \cdot Nr)$, where Nr is the maximum number of non-zeros per row.

Implementation

Inputs:

- *row_index*: the row vector of the COO format.
- *col_index*: the column vector of the COO format.
- *out_degree*: vector containing the out degree of every page.

```
def power_method(row_index, column_index, out_degree, max_iter=100, tol=1.0e-8,
     m = 0.15):
    # Finds the dangling nodes
    dangling_mask = (out_degree == 0)

    # x guess vector initialization
    n_pages = out_degree.shape[0]
    x = np.ones(n_pages, dtype=np.float64) / n_pages

    # m*S
    randomness = m / n_pages
    e_current = 0

    print("Starting page rank:")
    start_time = time.time()
```

Implementation

Main cycle:

```
    for i in range(0, max_iter):
        x_prev = x.copy()

        # It's like doing A*x since that Aij = 1/out_degree(ij)
        first_comp = x[column_index] / out_degree[column_index]

        # Effective implementation of x = (1 - m)*Ax
        x = np.zeros(n_pages, dtype=np.float64)
        np.add.at(x, row_index, first_comp)
        x *= 1 - m

        # Final computation x = x + mS with attention to the dangling nodes
        dangling_sum = np.sum(x_prev[dangling_mask])
        rank_correction = randomness + ((1 - m) * dangling_sum / n_pages)
        x += rank_correction

        e_current = (x_prev@x) / (x_prev@x_prev)

        err = np.sum(np.abs(x - x_prev))
        if err < tol:
            print(f"Convergenza raggiunta all'iterazione {i}.")
            break

    print(f"Finished in {time.time() - start_time:.2f} seconds")
    return e_current, x
```

Considerations

In this code we use the COO format to represent the link matrix A, since if we analyze a web with too many pages it becomes computationally unfeasible to create A.

But it is necessary that we keep in mind that the power method is supposed to work on a column-stochastic matrix like M defined as: $M = (1 - m)A + mS$.

So we modified the operations a bit so that it is more efficient but remains true to its premises.

Working

How it works:

Our aim is to work with M, since otherwise the power method wouldn't work, so we do: $x = Mx = (1 - m)Ax + mSx$.

First we identify the dangling nodes and then we create the intial guess x and the randomness factor which is m/num_pages which is equal to mS.

First Computation:
The first computation is $x = Ax$ and can be divided in two steps.

Working

Step 1. $first_comp = x[column_index]/out_degree[column_index]$
Let's analyze $x = Ax$, each $A_{ij} = 1/out_degree[j]$ but if j is a dangling node then $A_{ij} = 0$.
Now let's look at the i element of x:

$$x[i] = \sum_j A_{ij}x[j] = \sum_{j \to i} x[j]/out_degree[j]$$

So with this line of code we compute the rank distributed from the page j to page i.

Note that this step focuses only on the non dangling nodes, since their contribute will be addressed later.

Working

Step 2. $np.add.at(x, row_index, first_comp)$
We add to the new x the result obtained in the correct positions, so that each page i can benefit from the different pages pointing to it.

Second Computation: The second computation regards $x = x + mS$ with attention to the dangling nodes.

If we summed mS with no checks the column of the dangling nodes, which sum = 0, would all have m/n and its sum = m breaking the column-stochasticity of M, so in those columns we sum only 1/n.

$$mSx = \sum_{nd} \frac{m}{n} x_{nd} + \sum_{d} \frac{1}{n} x_d = \frac{1}{n} \left(m \sum_{nd} x_{nd} + \sum_{d} x_d \right)$$

Working

But since $\sum x = 1$ we can denote $\sum_{nd} x_{nd} = 1 - \sum_d x_d$, substituting we get:

$$mSx = \frac{1}{n}\left(\sum_d x_d + m\left(1 - \sum_d x_d\right)\right) =$$
$$= \frac{1}{n}\left(\sum_d x_d + m - m\sum_d x_d\right) =$$
$$= \frac{1}{n}\left(m + (1-m)\sum_d x_d\right) = \frac{m}{n} + \frac{1-m}{n}\sum_d x_d$$

Working

Finally we compute the Rayleigh Quotient as:

$$\lambda^{(m+1)} = x^{(m)} x^{(m+1)} = \frac{x^{(m)} A x^{(m)}}{\left(x^{(m)}\right)^T \cdot x^{(m)}}$$

If we look at the stop condition, we can see that we do not check the convergence of λ, because it is assured due to how the algorithm is built and works, we check for the convergence of x.

This allows us to impose the tolerance directly on the guess vector, avoiding a premature convergence of λ.

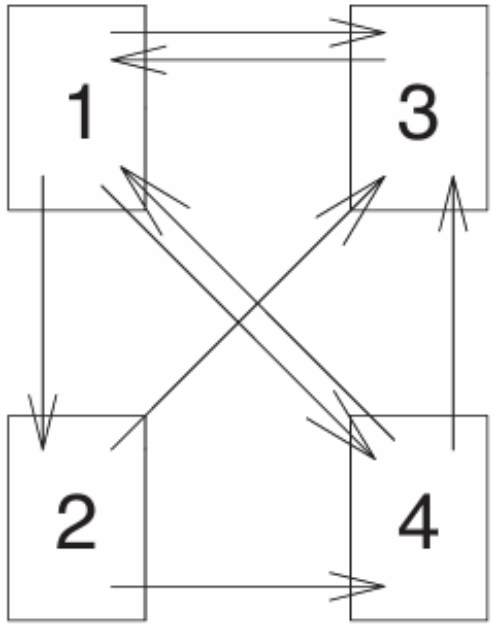
Results

In this section we will compare our method against np.linalg.eig, with the objective to see:

- Wether our method actually converges.
- Which value it converges to and if results are plausible.
- How much accurate our method is compared to np.linlg.eig.
- How much time is necessary for both methods.

(We will use m = 0.15 which is also Google's choice)

Graph 1



Matrices

$$A = \begin{bmatrix} 0 & 0 & 1 & 0.5 \\ 0.333 & 0 & 0 & 0 \\ 0.333 & 0.5 & 0 & 0.5 \\ 0.333 & 0.5 & 0 & 0 \end{bmatrix} \quad S = \begin{bmatrix} 0.25 & 0.25 & 0.25 & 0.25 \\ 0.25 & 0.25 & 0.25 & 0.25 \\ 0.25 & 0.25 & 0.25 & 0.25 \\ 0.25 & 0.25 & 0.25 & 0.25 \end{bmatrix}$$

$$M = \begin{bmatrix} 0.0375 & 0.0375 & 0.8875 & 0.4625 \\ 0.320833 & 0.0375 & 0.0375 & 0.0375 \\ 0.320833 & 0.4625 & 0.0375 & 0.4625 \\ 0.320833 & 0.4625 & 0.0375 & 0.0375 \end{bmatrix}$$

Results

Eigenvectors of M:

- np.linalg.eig:
[0.36815068 0.14180936 0.28796163 0.20207834]

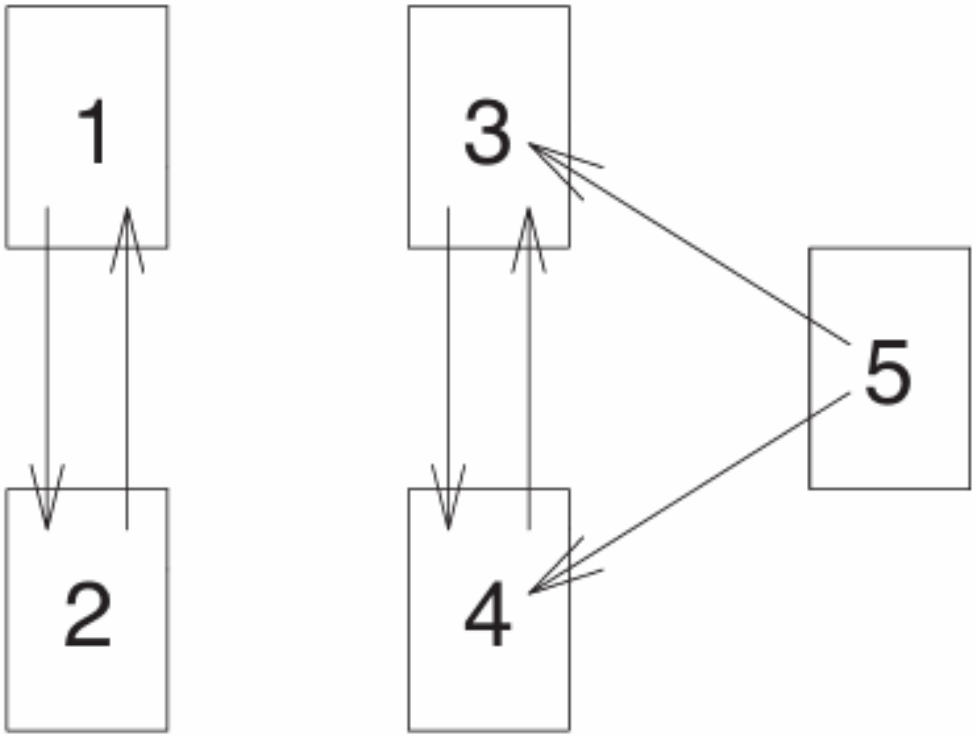
- Power Method:
[0.36815068 0.14180936 0.28796163 0.20207834]

Error between Eigenvectors: 1.7445149539871232e-09
Eigenvalue with power method: 0.9999999997656641

Computation time np.linalg.eig: $= 1$ ms
Computation time power method: $= 1$ ms

The power method stopped for err $<$ tol at iteration number $= 23$

Graph 2



Matrices

$$A = \begin{bmatrix} 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0.5 \\ 0 & 0 & 1 & 0 & 0.5 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix} \quad S = \begin{bmatrix} 0.2 & 0.2 & 0.2 & 0.2 & 0.2 \\ 0.2 & 0.2 & 0.2 & 0.2 & 0.2 \\ 0.2 & 0.2 & 0.2 & 0.2 & 0.2 \\ 0.2 & 0.2 & 0.2 & 0.2 & 0.2 \\ 0.2 & 0.2 & 0.2 & 0.2 & 0.2 \end{bmatrix}$$

$$M = \begin{bmatrix} 0.03 & 0.88 & 0.03 & 0.03 \\ 0.88 & 0.03 & 0.03 & 0.03 \\ 0.03 & 0.03 & 0.03 & 0.455 \\ 0.03 & 0.03 & 0.88 & 0.455 \\ 0.03 & 0.03 & 0.03 & 0.03 \end{bmatrix}$$

Results

Eigenvectors of M:

- np.linalg.eig:
[0.2 0.2 0.285 0.285 0.03]

- Power Method:
[0.2 0.2 0.285 0.285 0.03]

Error between eigenvectors: 1.7627658722935884e-15
Eigenvalue with power method: 1

Computation time np.linalg.eig: ≈ 1 ms
Computation time power method: ≈ 1 ms

The power method stopped for err $<$ tol at iteration number ≈ 1

Hollins database Results

Eigenvectors of $M_{hollins}$:

- np.linalg.eig:
[5.80584118e-05 1.98787505e-02 1.12567975e-04 ...
6.89072123e-05 6.89072123e-05 1.75820619e-04]

- Power method:
[5.80584153e-05 1.98787511e-02 1.12567981e-04 ...
6.89072159e-05 6.89072159e-05 1.75820618e-04]

Error between eigenvectors: 2.7296176611360708e-09
Eigenvalue with power method: 0.99999996165265

Computation time np.linalg.eig: $\approx$ 57 s
Computation time power method: $\approx$ 25 ms

The power method stopped for err $<$ tol at iteration number $\approx$ 83

Most Important Pages

Top 10 pages:

Page	Score	Pointed by	Points to
2	0.019878750637885	829	25
37	0.009287620279790	414	14
38	0.008610392961889	435	31
61	0.008065030706612	390	10
52	0.008026564887810	417	11
43	0.007164642979337	377	15
425	0.006582780807498	87	77
27	0.005989213098725	168	12
28	0.005571736100496	284	24
4023	0.004452468200947	54	4

Least Important Pages

Bottom 10 pages:

Page	Score	Pointed by	Points to
2021	5.843605110191e-5	1	0
2020	5.843605110191e-5	1	0
2019	5.843605110191e-5	1	0
2018	5.843605110191e-5	1	0
2017	5.843605110191e-5	1	0
2016	5.843605110191e-5	1	0
2015	5.843605110191e-5	1	0
2014	5.843605110191e-5	1	0
51	5.805841501852e-5	0	26
1	5.805841501852e-5	0	24

Convergence

In all 3 cases our power method converges to the same solution of np.linalg.eig, with excellent precision too, it varies accordingly to the tolerance selected.

If we take a look at the results, we can understand that logically they are correct, the page which is most pointed to, in general, gets the higher score.

In all cases the method also converges to λ_1 equal to 1.

Computation cost

In the first 2 scenarios our method and np.linalg.eig return a solution in practically the same amount of time: ≈ 1 ms.

With the Hollins dataset instead we can see the difference between the computation time, since now in the problem there are more or less 6000 pages, rather than 4/5 like in the first two problems.

This difference is caused by our best use of the sparsity of the matrix A, allowing us to obtain a computation time ≈ 2200 times faster than the one obtained with linalg.eig, that's three orders of magnitude!

Notes

Since we have ascertained that our method works and converges to a right result, we will be using it in the solving of the following exercises without comparing it to np.linalg.eig anymore.

However in some exercises we will still use np.linalg.eig to compute all eigenvalues and eigenvectors, as it is necessary to the solving of the exercise.

We will also assume there are no dangling nodes in every exercise.

Exercise 1

Task:

Suppose the people who own page 3 in the web of Figure 1 are infuriated by the fact that its importance score, computed using formula $x_k = \sum_{j \in L_k} \frac{x_j}{n_j}$, is lower than the score of page 1.

In an attempt to boost page 3's score, they create a page 5 that links to page 3; page 3 also links to page 5.

Does this boost page 3's score above that of page 1?

Exercise 1

Solution:
Page 3 owners added page 5 which only points to page 3 and is pointed to only by page 3. So we have:

Links:

$$p_1 \to p_2, p_3, p_4$$
$$p_2 \to p_3, p_4$$
$$p_3 \to p_1, p_5$$
$$p_4 \to p_1, p_3$$
$$p_5 \to p_3$$

Importance score:

$$x_1 = x_3/2 + x_4/2$$
$$x_2 = x_1/3$$
$$x_3 = x_1/3 + x_2/2 + x_4/2 + x_5$$
$$x_4 = x_1/3 + x_2/2$$
$$x_5 = x_3/2$$

So we can compute the eigenvector.

Exercise 1

$$A = \begin{bmatrix} 0 & 0 & 0.5 & 0.5 & 0 \\ 0.33 & 0 & 0 & 0 & 0 \\ 0.33 & 0.5 & 0 & 0.5 & 1 \\ 0.33 & 0.5 & 0 & 0 & 0 \\ 0 & 0 & 0.5 & 0 & 0 \end{bmatrix}$$

Eigenvector:
[0.24489796 0.08163265 0.36734694 0.12244898 0.18367347]

So we can see that adding page 5 does indeed boost page 3 score above page 1's, making it the most important page.

Exercise 3

Task:
Add a link from page 5 to page 1 in the web of Figure 2.

The resulting web, considered as an undirected graph, is connected.

What is the dimension of $V_1(A)$?

Exercise 3

Solution:

Links:

$$p_1 \to p_2$$
$$p_2 \to p_1$$
$$p_3 \to p_4$$
$$p_4 \to p_3$$
$$p_5 \to p_1, p_3, p_4$$

Importance score:

$$x_1 = x_2 + x_5/3$$
$$x_2 = x_1$$
$$x_3 = x_4 + x_5/3$$
$$x_4 = x_3 + x_5/3$$
$$x_5 = 0$$

Exercise 3

$$A = \begin{bmatrix} 0 & 1 & 0 & 0 & 0.33 \\ 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0.33 \\ 0 & 0 & 1 & 0 & 0.33 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

Eigenvalues:
$[1 \quad -1 \quad 1 \quad -1 \quad 0]$
Since we have 2 eigenvalues equals to 1, we will have two eigenvectors that we can consider as importance score:
1. $[0.5 \quad 0.5 \quad 0 \quad 0 \quad 0]$ 2. $[0 \quad 0 \quad 0.5 \quad 0.5 \quad 0]$
So we can conclude that $V_1(A) = 2$.

Exercise 5

Task:
Prove that in any web the importance score of a page with no backlinks is zero.

Proof:
The importance score x_i of page i is given by the sum of the other pages' score, those who are pointing to page i, over the number of their outgoing links.

If page i has no backlinks that means that no pages are pointing to page i, making the sum, and so its score, equal to 0.

Exercise 6

Task:
Implicit in our analysis up to this point is the assertion that the manner in which the pages of a web W are indexed has no effect on the importance score assigned to any given page, prove this.

Solution:
Let W contains n pages, each page assigned an index 1 through n, and let A be the resulting link matrix, note A is a square matrix.
Suppose we then transpose the indices of pages i and j.
We do this having $\tilde{A} = PAP$ where P is the elementary matrix obtained by transposing rows i and j of the n×n identity matrix.

Exercise 6

$$A = \begin{bmatrix} a_{11} & \dots & a_{1n} \\ \dots & \dots & \dots \\ a_{i1} & \dots & a_{in} \\ a_{j1} & \dots & a_{jn} \\ \dots & \dots & \dots \\ a_{n1} & \dots & a_{nn} \end{bmatrix} \quad P = \begin{bmatrix} 1 & \dots & \dots & \dots & \dots & 0 \\ \dots & \dots & \dots & \dots & \dots & \dots \\ 0 & \dots & 0 & 1 & \dots & 0 \\ 0 & \dots & 1 & 0 & \dots & 0 \\ \dots & \dots & \dots & \dots & \dots & \dots \\ 0 & \dots & \dots & \dots & \dots & 1 \end{bmatrix}$$

The operation A = PA has the effect of swapping row i with row j, instead the operation A = AP has the effect of swapping column i and j.
If Ax = x then x applied to $\tilde{A} \rightarrow \tilde{A}x = x$ will give us the same eigenvector, just with x_i and x_j in swapped positions, therefore not changing anything in the importance score.

Exercise 7

Task:
Prove that if A is an n x n column-stochastic matrix and $0 \leq m \leq 1$, then $M = (1 - m)A + mS$ is also a column-stochastic matrix.

Proof:
If we develop the sum for M each element will be in the form $M_{ij} = (1 - m) \cdot A_{ij} + \frac{m}{n} = A_{ij} - m \cdot A_{ij} + m$. Now if we compute the sum on the column we obtain:

$$\sum_{i=1}^{n}(A_{ij} - m \cdot A_{ij} + \frac{m}{n}) = \sum_{i=1}^{n} A_{ij} - \sum_{i=1}^{n} m \cdot A_{ij} + \sum_{i=1}^{n} \frac{m}{n}$$

Exercise 7

But $\sum_{i=1}^{n} A_{ij} = 1$ for A is column-stochastic, $\sum_{i=1}^{n} m \cdot A_{ij} = m$ for the same reason and $\sum_{i=1}^{n} \frac{m}{n} = m$ so:

$$\sum_{i=1}^{n} A_{ij} - \sum_{i=1}^{n} m \cdot A_{ij} + \sum_{i=1}^{n} \frac{m}{n} = 1 - m + m = 1$$

This proves that $M = (1 - m)A + mS$ is column-stochastic.

Exercise 8

Task:
Show that the product of two column-stochastic matrices is also column-stochastic.
Proof:
If we take two matrices A size m x n and B size n x z both column-stochastic and we compute the product S = AB, analyzing element for element for the first column we get:

$$s_{00} = a_{00} \cdot b_{00} + a_{01} \cdot b_{10} + ... + a_{0n} \cdot b_{n0}$$
$$s_{10} = a_{10} \cdot b_{00} + a_{11} \cdot b_{10} + ... + a_{1n} \cdot b_{n0}$$
$$...$$
$$s_{m0} = a_{n0} \cdot b_{00} + a_{n1} \cdot b_{10} + ... + a_{nn} \cdot b_{n0}$$

Exercise 8

It is easy to understand that the situation for the other columns is the same. Now let's do the sum of the first column, we can see that if we isolate every b element we get something like this:

$$\sum_{i=1}^{m} s_{i0} = b_{00}(a_{00} + ... + a_{n0}) + ... + b_{n0}(a_{0n} + ... + a_{nn}) =$$

$$= b_{00} \sum_{i=1}^{n} a_{i0} + ... + b_{n0} \sum_{i=1}^{n} a_{in}$$

But each sum of the a-elements is the sum along the column in the matrix A, therefore each sum equals to 1 per definition, since A is column-stochastic.

Exercise 8

So we have:

$$\sum_{i=1}^{m} s_{i0} = b_{00} + ... + b_{n0} = \sum_{i=1}^{n} b_{i0}$$

But this is also the sum of the b-elements along the columns 0 of the matrix B, giving us 1 as well. The same reasoning can be applied to the other columns of S.

This proves that the product of two column-stochastic matrices, even with different sizes, is always a column-stochastic matrix.

Exercise 9

Task:
Show that a page with no backlinks is given importance score $\frac{m}{n}$ by formula $x = (1 - m)Ax + ms$.

Proof:
As demonstrated in exercise 5 a page with no backlinks is given the importance score of 0 if we just consider A, but this remains the same if we do (1 - m)A since this does not change the 0-elements of the matrix.
Now let's compute:

$$x = (1 - m)Ax + mS = 0 + ms \rightarrow x = \frac{m}{n}$$

Exercise 10

Task:
Suppose that A is the link matrix for a strongly connected web of n pages (any page can be reached from any other page by following a finite number of links).

Show that dim(V1(A)) = 1.

Proof:

Let $(A^k)_{ij}$ denote the (i, j)-entry of A^k, $A_{ij} > 0$ if there's a link from page j to page i and therefore can be reached in one step.

The same reasoning can be applied to A^2, since $(A^2)_{ij} = \sum_{x=1}^{n} A_{ix} Axj$, if $(A^2)_{ij} > 0$ then there must be a x such that both $(A^2)_{ix}$ and $(A^2)_{xj}$ are positive.

Exercise 10

This reasoning can be extended to a general k, saying that if $(A^k)_{ij} > 0$ then page j can reach page i in exactly k steps. Let's focus on the sum $\tilde{A} = (I + A + A^2 + ... + A^p)$, now if we take $\tilde{A}_{ij}$ and it's > 0 then it means that page j can reach page i in p steps or less.

Note that if we take p = 0 we obtain $A^0 = I$ which makes sense because every page can reach itself in 0 steps.

We can observe that:

$$\tilde{A} = I + A + A^2 + ... + A^{n-1} = \sum_{k=0}^{n-1} A^k$$

Exercise 10

$\tilde{A}$ is a positive matrix, because, as definition, A represents a strongly connected web, therefore page i can be reached in a finite number of steps, which in the worst case scenario is n-1, this ensures that in the sum each entry $\tilde{A}_{ij} > 0$.

As proven in exercise 8 a product between two column-stochastic matrices always give a column-stochastic matrix, therefore A^k will always be column-stochastic.

So if we define $B = \frac{1}{n} \cdot (I + A + A^2 + ... + A^{n-1})$ and we focus on the columns, we can observe that at every step in the sum in each column we add a total of 1, this is why we need to divide by n, so that we normalize the result making B also a positive column-stochastic matrix.

Exercise 10

In this way, with Lemma 3.2, we can prove that $V_1(B) = 1$.

Now let's focus on the eigenvector x of A:

$$Ax = \lambda x \rightarrow A^k x = \lambda^k x \rightarrow Bx = \frac{1}{n} \cdot (I + A + A^2 + ... + A^{n-1})x$$

$$Bx = \frac{1}{n} \cdot (1 + \lambda + \lambda^2 + ... + \lambda^{n-1})x$$

We can conclude then that if $x \in V_1(A)$ then $x \in V_1(B)$, but since $V_1(B) = 1$ then it must be that $V_1(A) = 1$.

Exercise 11

Task:

Consider again the web in Figure 2.1, with the addition of a page 5 that links to page 3, where page 3 also links to page 5.
Calculate the new ranking by finding the eigenvector of M (corresponding to $\lambda = 1$) that has positive components summing to one. Use m $= 0.15$.

Solution:

$$A = \begin{bmatrix} 0 & 0 & 0.5 & 0.5 & 0 \\ 0.33 & 0 & 0 & 0 & 0 \\ 0.33 & 0.5 & 0 & 0.5 & 1 \\ 0.33 & 0.5 & 0 & 0 & 0 \\ 0 & 0 & 0.5 & 0 & 0 \end{bmatrix} \quad S = \begin{bmatrix} 0.2 & 0.2 & 0.2 & 0.2 & 0.2 \\ 0.2 & 0.2 & 0.2 & 0.2 & 0.2 \\ 0.2 & 0.2 & 0.2 & 0.2 & 0.2 \\ 0.2 & 0.2 & 0.2 & 0.2 & 0.2 \\ 0.2 & 0.2 & 0.2 & 0.2 & 0.2 \end{bmatrix}$$

Exercise 11

$$M = (1-m)A + mS = \begin{bmatrix} 0.03 & 0.03 & 0.455 & 0.455 & 0.03 \\ 0.313 & 0.03 & 0.03 & 0.03 & 0.03 \\ 0.313 & 0.455 & 0.03 & 0.455 & 0.88 \\ 0.313 & 0.455 & 0.03 & 0.03 & 0.03 \\ 0.03 & 0.03 & 0.455 & 0.03 & 0.03 \end{bmatrix}$$

Eigenvector computed with the power method:
[0.2371406 0.0971898 0.3488941 0.1384955 0.1782800]

So this gives us a ranking of:

1. Page 3.
2. Page 1.
3. Page 5.
4. Page 4.
5. Page 2.

Exercise 12

Task:
Add a sixth page that links to every page of the web in the previous exercise, but to which no other page links.
Rank the pages using A, then using M with m = 0.15, and compare the results.

Solution:

$$A = \begin{bmatrix} 0 & 0 & 0.5 & 0.5 & 0 & 0.2 \\ 0.33 & 0 & 0 & 0 & 0 & 0.2 \\ 0.33 & 0.5 & 0 & 0.5 & 1 & 0.2 \\ 0.33 & 0.5 & 0 & 0 & 0 & 0.2 \\ 0 & 0 & 0.5 & 0 & 0 & 0.2 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} \qquad S = 0.166 \cdot 1^{6x6}$$

Exercise 12

$$M = \begin{bmatrix} 0.025 & 0.025 & 0.45 & 0.45 & 0.025 & 0.195 \\ 0.3083 & 0.025 & 0.025 & 0.025 & 0.025 & 0.195 \\ 0.3083 & 0.45 & 0.025 & 0.45 & 0.875 & 0.195 \\ 0.3083 & 0.45 & 0.025 & 0.025 & 0.025 & 0.195 \\ 0.025 & 0.025 & 0.45 & 0.025 & 0.025 & 0.195 \\ 0.025 & 0.025 & 0.025 & 0.025 & 0.025 & 0.025 \end{bmatrix}$$

Eigenvector computed with A:
[0.244898 0.081633 0.367347 0.122449 0.183674 0]

Eigenvector computed with M
[0.231212 0.094760 0.340172 0.135033 0.173823 0.025]

Exercise 12

As we can see the addiction of page 6 doesn't really change the eigenvector that much, the pages' score barely changes and the most important pages are:

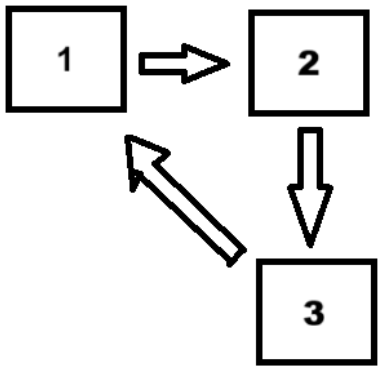
- Page 3
- Page 1
- Page 5
- Page 4
- Page 2
- Page 6

The interesting thing is that page 6's score is no longer 0, this is because of the addiction of mS which represent the randomness of the system.

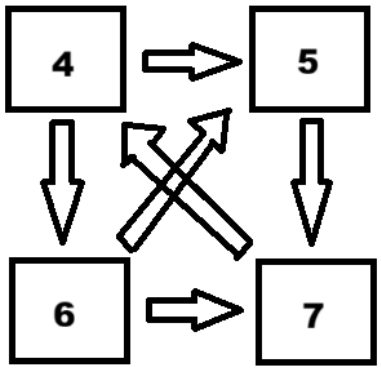
Exercise 13

Task:
Construct a web consisting of two or more subwebs and determine the ranking given by formula M =(1 - m)A + mS.

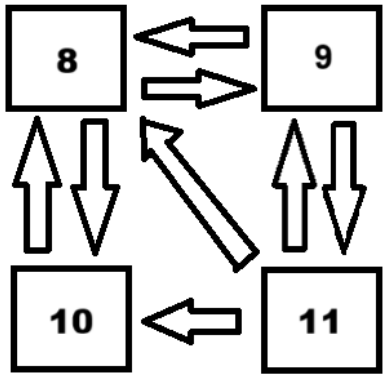
Subweb 1



Subweb 2



Subweb 2



Exercise 13

Eigenvector found with power method:
[0.09090909 0.09090909 0.09090909
0.10807628 0.08488552 0.05956878 0.11110578
0.13894645 0.08703239 0.08703239 0.05062513]

Let's analyze the different subwebs:

- **Subweb 1**, pages 1 to 3:
  Their importance score all are equal, that's because every page points to another one in a circle.
- **Subweb 2**, pages 4 to 7:
  The ranking is page 4, page 7, page5 and page 6.
- **Subweb 3**, pages 8 to 11:
  The ranking is page 8, then page 9 and page 10 equal and, lastly, page 11.

Exercise 14

Task:
For the web in Exercise 11, compute the values of $\|M^k x_0 - q\|_1$ and $\frac{\|M^k x_0 - q\|_1}{\|M^{k-1} x_0 - q\|_1}$, for k = 1, 5, 10, 50, using an initial guess x_0 not too close to the actual eigenvector q (so that you can watch the convergence).
Determine $c = max_{1 \leq j \leq n} |1 - 2 \cdot min_{1 \leq i \leq n} M_{ij}|$ and the absolute value of the second largest eigenvalue of M.

Solution:
For this exercise we will use $x_0 = [1.0 \quad 0 \quad 0 \quad 0 \quad 0]^T$. Firstly we'll compute M, then the eigenvector associated to $\lambda_1 = 1$, using the power method, and in the end $\|M^k x_0 - q\|_1$ and $\frac{\|M^k x_0 - q\|_1}{\|M^{k-1} x_0 - q\|_1}$, so we can finish with the calculations of c and $|\lambda_2|$.

Exercise 14

With the power method we get the following eigenvector:
$v = [0.2371406 \quad 0.0971898 \quad 0.3488941 \quad 0.1384955 \quad 0.1782799]$

With a cycle we obtain the following results:

K	$\|M^k x_0 - q\|_1$	$\frac{\|M^k x_0 - q\|_1}{\|M^{k-1} x_0 - q\|_1}$
1	0.7819626529	0.51252
5	0.0068714107	0.32271
10	0.0003043717	0.52453
50	0.0000000000	0.62475

So we can see how $\lim_{k\to\infty} \|M^k x_0 - q\|_1 = 0$ and how $\lim_{k\to\infty} \frac{\|M^k x_0 - q\|_1}{\|M^{k-1} x_0 - q\|_1} = |\lambda_2|$.

Exercise 14

Regarding c we know that, with M built in that way, $0 \leq c \leq 1 - 2m/n$ so let's calculate its upper bound:

$$c = 1 - 2m/n = 1 - 2 \cdot 0.15/5 = 0.94$$

Then let's calculate our c:

$$c = max_{1 \leq j \leq n} |1 - 2 \cdot min_{1 \leq i \leq n} M_{ij}| = 0.94$$

We can observe that the two results matches.

Having chosen an m $= 0.15$ this allows us also quite a fast convergence time, in fact, if we look at the code, we can see that a good approximation of $|\lambda_2|$ is present at circa iteration 15.

Exercise 16

Task:
Consider the link matrix:

$$A = \begin{bmatrix} 0 & 0.5 & 0.5 \\ 0 & 0 & 0.5 \\ 1 & 0.5 & 0 \end{bmatrix}$$

Show that: $M = (1 - m) \cdot A + m \cdot S$ (all $S_{ij} = 1/3$) is not diagonalizable for $0 \leq m < 1$.

Solution:
To show that M is diagonalizable we must prove that for every eigenvalue the algebraic multiplicity equals the geometric multiplicity.

Exercise 16

$$S_{ij} = \frac{1}{3} \qquad M = \begin{bmatrix} \frac{m}{3} & \frac{1-m}{2} + \frac{m}{3} & \frac{1-m}{2} + \frac{m}{3} \\ \frac{m}{3} & \frac{m}{3} & \frac{1-m}{2} + \frac{m}{3} \\ 1 - m + \frac{m}{3} & \frac{1-m}{2} + \frac{m}{3} & \frac{m}{3} \end{bmatrix}$$

Let's calculate the characteristic polynomial as $p(\lambda) = det(M - \lambda I)$:

$$D = M - \lambda I = \begin{bmatrix} \frac{m}{3} - \lambda & \frac{1-m}{2} + \frac{m}{3} & \frac{1-m}{2} + \frac{m}{3} \\ \frac{m}{3} & \frac{m}{3} - \lambda & \frac{1-m}{2} + \frac{m}{3} \\ 1 - m + \frac{m}{3} & \frac{1-m}{2} + \frac{m}{3} & \frac{m}{3} - \lambda \end{bmatrix}$$

$$p(\lambda) = det(D) = (\lambda - 1) \cdot (\lambda - \tfrac{m-1}{2})^2$$

Exercise 16

As we can see the eigenvalues $(1, \frac{m-1}{2})$ have algebraic multiplicity of (1, 2), let's see if the geometric multiplicity is also (1, 2), otherwise M is not diagonalizable.

- $\lambda = 1$
  Since $1 \leq mg \leq ma$ and ma $= 1$ then mg also $= 1$.
- $\lambda = \frac{m-1}{2}$
  We can compute the geometric multiplicity as:

$$mg = dim - rank(D) = 3 - rank(M - \frac{m-1}{2}I)$$

$$D = \begin{bmatrix} \frac{3-m}{6} & \frac{3-m}{6} & \frac{3-m}{6} \\ \frac{m}{3} & \frac{3-m}{6} & \frac{3-m}{6} \\ \frac{3-2m}{3} & \frac{3-m}{6} & \frac{3-m}{6} \end{bmatrix}$$

Exercise 16

If we look at the columns of D we can see that column 2 and 3 are identical and therefore L.D., if we compare instead column 1 and 2 we can see that they are L.D if and only if m $= 1$.

But in our premise we say that it must be $m < 1$, therefore column 1 and column 2 must be L.I. giving us $rank(D) = 2$ and $mg = 3 - rank(D) = 1$ which is different from $ma = 2$.

We can conclude that, for the link matrix A and $0 \leq m < 1$, M is not diagonalizable.

Exercise 17

Task:
How should the value of m be chosen? How does this choice affect the rankings and the computation time?

Solution:
There's not exactly a right choice for m, its value can be decided by analyzing different approaches:

- Matrix M and rankings
- Computation time

Exercise 17

- Matrix m and rankings

One way it's to analyze M and how it's built, we know that $M = (1 - m)A + mS$, if we choose a low value of m this means that we are giving more importance to the link matrix A and the rankings would depend heavily on that, instead if we choose a higher value it means that the matrix A is not that important giving us a ranking more uniform.

Let's take the example of the web-surfer, the surfer starts at a random page and randomly moves to another one according to the following procedure: if the surfer is currently at a page with r outgoing links, he either randomly chooses anyone of these links with probability $\frac{1-m}{r}$ or he jumps to any randomly selected page on the web, each with probability $\frac{m}{n}$.

Exercise 17

Let's analyze the different scenarios:

- **Lower m**

  If we choose a low value for m, it gives us a higher probability of the surfer to follow the links through the pages.

  Be aware that a really low m might create some trouble in the randomness of a web search, if m is 0 we end up in the case where the surfer only follows the pages' links, but, if the web is not connected, it might get trapped in a circle following the same links, leaving some pages unvisited and killing the randomness of the process.

Exercise 17

- **Higher m**

  On the other hand if we assign m a high value it significantly lowers the importance of the link matrix A, if for example $m = 1$ then M = S completely disregarding the matrix A.

  In the example of the web surfer this means that the probability of it jumping to a random web page, rather than following links between the pages, is very high.

  This might cause problems in the opposite way as before, a high m might completely ignore the links between pages making the overall structure useless.

exercise 17

- **Computation time**

Another way is to consider the computation time, since it is demonstrated that the power method converges asymptotically according $\|Mx_k - q\|_1 \approx |\lambda_2| \cdot \|x - q\|_1$, this means that the lower $|\lambda_2|$ is, the faster the method converges.

But it's also known that $|\lambda_2| \leq 1 - m$ and this gives us two scenarios over the value of m:

- **High m**:
  The method will converge really fast.
- **Low m**:
  The method will converge really slow.

Exercise 17

So, how should we choose m?

If we go for a high value we get a faster computation time but also a higher randomness of the web.

If we go for a lower value we get a slower computation time but we give more importance to A.

A good compromise between speed and randomness is, for example, the value originally used by google, which is $m = 0.15$.

But this is not the only option, one can choose m accordingly, depending on the results that want to be achieved.